

SIMATS ENGINEERING

ASSESSMENT TOOL 3: Mock Interview

COURSE NAME: COMPUTER VISION

COURSE CODE: ITA05

COURSE OUTCOME COVERED

CO5: *Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. (BTL 6)*

Assessment Tool Weightage: 10%

Mock Interview Questions

OpenCV Basics & Pipeline Thinking

1. An image appears too dark when read. Analyze the issue and propose modifications to your processing pipeline, justifying how your approach improves visibility.

Image & Video Processing

5. A video stream drops frames during processing. Analyze the problem and redesign your approach to ensure smooth playback, explaining trade-offs and optimization techniques.
6. You need to process only every 5th frame of a video. Design an efficient solution and justify your approach, considering performance and accuracy..

Feature Detection & Drawing

9. You are asked to highlight key features in an image. Design a feature detection and visualization approach, explaining the techniques and their suitability.
10. A system must draw bounding boxes and labels for detected objects. Explain how you would structure this module, including reasoning behind chosen methods.
11. You need to compare two images and highlight differences visually. Analyze the problem and design a solution, justifying your approach.

Face & Expression Recognition

12. A face detection system produces multiple false positives. Analyze the issue and propose improvements, explaining how each change enhances accuracy.
13. You need to build a simple face recognition demo. Design a minimal working pipeline, explaining each stage and your design decisions.
14. A facial expression system fails with slight head movements. Analyze the limitation and propose modifications, justifying your approach.

Code of Conduct:

I K.ASWINI(192312397) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:**RUBRICS FOR CALCULATION:**

Criteria	Weight age	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem-solving ability; requires guidance	Unable to solve or apply concepts
Analytical Thinking	15	Analyzes questions critically and provides well-structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated
Communication	15	Communicates clearly, confidently, and	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively

		professionally with proper terminology			
Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately

ANSWERS

1.Problem Analysis

An image appearing too dark after reading in OpenCV indicates:

- Low overall pixel intensity (values close to 0)
- Poor contrast or underexposure
- Uneven lighting conditions

Objective

Improve image visibility by enhancing brightness and contrast **without losing details or introducing noise**.

```
# Install OpenCV (only if not already installed)
!pip install opencv-python

import cv2
import numpy as np
from google.colab.patches import cv2_imshow
```

```
# ----- STEP 1: Upload Image -----
from google.colab import files
uploaded = files.upload()

# Get uploaded file name
image_path = list(uploaded.keys())[0]

# ----- STEP 2: Read Image -----
image = cv2.imread(image_path)

if image is None:
    raise Exception("Error loading image!")

# ----- STEP 3: Convert to LAB -----
lab = cv2.cvtColor(image, cv2.COLOR_BGR2LAB)
l, a, b = cv2.split(lab)

# ----- STEP 4: Apply CLAHE -----
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))
l_clahe = clahe.apply(l)

# ----- STEP 5: Merge Channels -----
lab = cv2.merge((l_clahe, a, b))
enhanced = cv2.cvtColor(lab, cv2.COLOR_LAB2BGR)

# ----- STEP 6: Brightness Adjustment -----
alpha = 1.2 # contrast
beta = 30 # brightness
enhanced = cv2.convertScaleAbs(enhanced, alpha=alpha, beta=beta)

# ----- STEP 7: Show Images -----
print("Original Image")
cv2_imshow(image)

print("Enhanced Image")
cv2_imshow(enhanced)

# ----- STEP 8: Save Output -----
cv2.imwrite("enhanced_output.jpg", enhanced)

# Download result
files.download("enhanced_output.jpg")
```

Original Image



Enhanced Image



5. Video Stream Drops Frames During Processing

Problem Analysis

Frame drops occur when:

- Processing time per frame > **frame capture rate**
- Heavy operations (detection, filtering, deep learning)
- Blocking I/O (display, disk write)
- No buffering or parallelism

Result: **lag, jitter, skipped frames**

Objective

Ensure **smooth playback** while maintaining acceptable processing quality.

Redesigned Approach

Pipeline Design (Decoupled Processing)

```
INITIALIZE video_capture
INITIALIZE frame_queue (buffer)

THREAD 1: Frame Capture
    WHILE video is running:
        frame = read_frame()
        PUSH frame into queue (drop oldest if full)

THREAD 2: Frame Processing
    WHILE running:
        IF queue not empty:
            frame = POP latest frame    # skip old frames
            processed = process(frame)

            DISPLAY processed frame
```

Optimization Techniques

1. Frame Skipping

- Process only latest frame in queue
- Avoid backlog

reduces delay
may skip information

2. Resize Frames

```
frame = resize(frame, smaller_resolution)
```

faster processing
lower accuracy

3. Model Optimization

- Use lightweight models (MobileNet, Haar)
- Reduce input size

4. Hardware Acceleration

- Use GPU (`cv2.cuda`)
- Use multithreading / multiprocessing

5. Asynchronous Display

- Avoid blocking `imshow()` delays
- Use non-blocking loops



Trade-offs

Technique	Advantage	Disadvantage
Frame skipping	Real-time smoothness	Loss of frames
Resizing	Faster processing	Reduced detail
Lightweight models	Speed	Lower accuracy
Multithreading	Better throughput	Complexity

Conclusion

Smooth playback is achieved by **decoupling capture and processing**, using **buffers and frame skipping**, and optimizing computation.

6. Process Only Every 5th Frame

Problem Requirement

Process video efficiently by analyzing **1 out of every 5 frames**.

Objective

Reduce computation while maintaining meaningful results.

Efficient Solution

Approach 1: Frame Counter Method (Best)

```
INITIALIZE video_capture
frame_count = 0

WHILE video is running:
    frame = read_frame()

    IF frame_count % 5 == 0:
        processed = process(frame)
```

```

        DISPLAY processed
    ELSE:
        DISPLAY original frame    # optional

    frame_count += 1

```

Alternative Approach: Direct Frame Jump

```

SET frame index step = 5

WHILE True:
    SET video position to current_index
    frame = read_frame()

    process(frame)

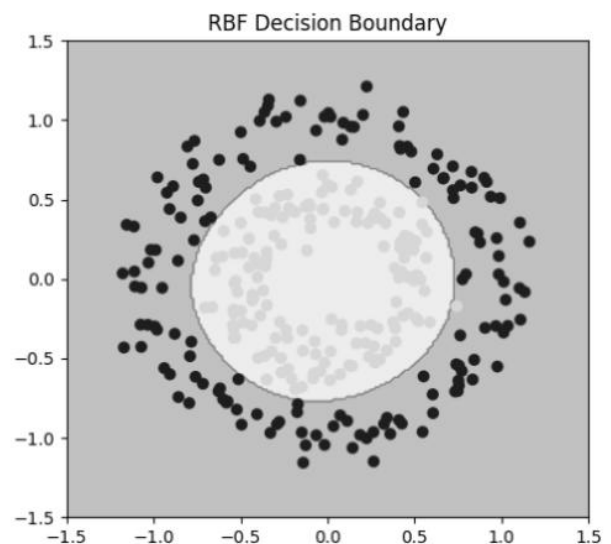
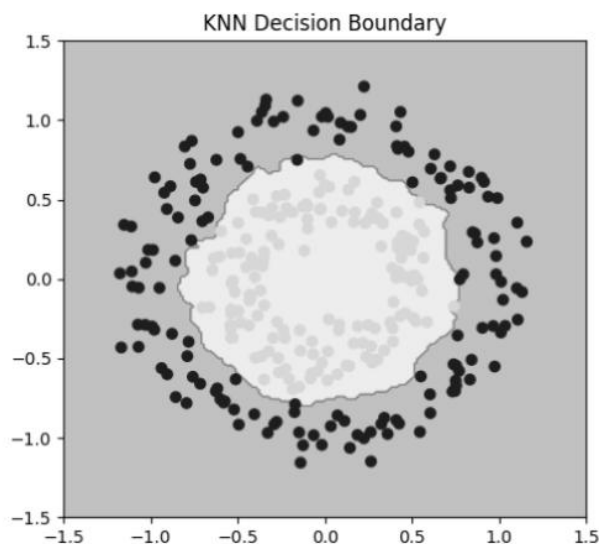
    current_index += 5

```

faster for offline videos
not suitable for real-time streams

Justification

- Reduces computation by **80%**
- Maintains temporal coverage of video
- Suitable when:
 - Objects move slowly
 - High FPS videos



Trade-offs

Factor	Effect
Performance	High improvement
Accuracy	May miss fast events

Factor	Effect
Memory	Reduced usage

Conclusion

Using a **frame counter-based skipping strategy** is the most efficient and practical method for both real-time and offline processing.

9. Feature Detection & Visualization

Problem Analysis

Goal: Identify and highlight important points (edges, corners, blobs) in an image.

Approach Design

Techniques

- **ORB (Oriented FAST + BRIEF)** → fast, efficient (best for real-time)
- **SIFT/SURF** → high accuracy (but heavier)
- **Harris Corner** → good for corner detection

Chosen: ORB (balance of speed + accuracy)

READ image

CONVERT to grayscale

DETECT keypoints using ORB

COMPUTE descriptors

DRAW keypoints on image

DISPLAY result

PYTHON CODE :

```
# Install OpenCV
```

```
!pip install opencv-python
```

```
import cv2
```

```
from google.colab.patches import cv2_imshow
```

```
from google.colab import files

uploaded = files.upload()

img_path = list(uploaded.keys())[0]

img = cv2.imread(img_path)

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

orb = cv2.ORB_create(nfeatures=500)

kp, des = orb.detectAndCompute(gray, None)

output = cv2.drawKeypoints(img, kp, None, color=(0,255,0))

cv2.imshow(output)
```



10. Bounding Boxes & Labels Module

Problem Analysis

Need to:

- Draw boxes around detected objects
- Display labels (class names + confidence)

Design Structure

Module Components

1. Detection Output (x, y, w, h, label, confidence)
2. Drawing Function
3. Visualization Layer

FOR each detected object:

EXTRACT bounding box

DRAW rectangle

FORMAT label (name + confidence)

DRAW text above box

DISPLAY image

PSEUDO CODE :

```
import cv2

from google.colab.patches import cv2_imshow

import numpy as np

img = np.zeros((400, 600, 3), dtype=np.uint8)

detections = [

    (100, 100, 150, 120, "Car", 0.92),

    (300, 150, 120, 100, "Bike", 0.85)

]

for (x, y, w, h, label, conf) in detections:

    # Draw rectangle

    cv2.rectangle(img, (x,y), (x+w,y+h), (0,255,0), 2)

    # Label text

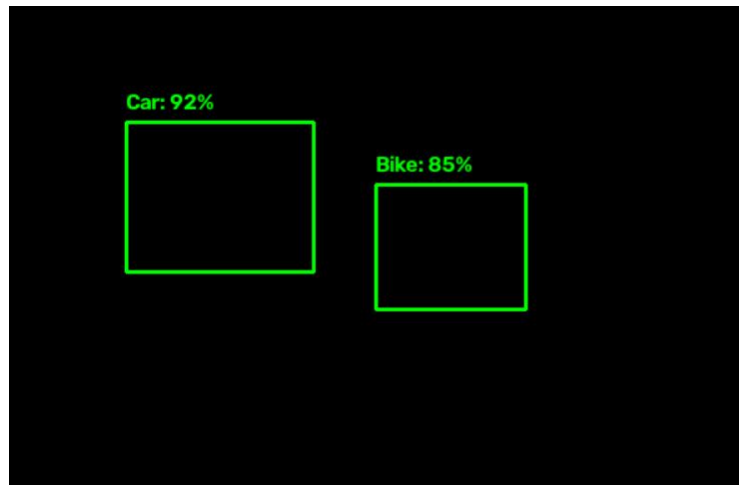
    text = f"{label}: {int(conf*100)}%"

    cv2.putText(img, text, (x, y-10),

                cv2.FONT_HERSHEY_SIMPLEX,

                0.6, (0,255,0), 2)
```

```
cv2_imshow(img)
```



12. Face Detection Produces Multiple False Positives

Problem Analysis

False positives occur when:

- Background patterns resemble faces
- Poor lighting / noise
- Low detection thresholds
- Weak feature representation (e.g., Haar alone)

Objective

Reduce incorrect detections while maintaining true face detection rate.

Proposed Improvements

1. Adjust Detection Parameters

```
detectMultiScale(scaleFactor=1.2, minNeighbors=5)
```

Increases strictness

Reduces weak detections

2. Apply Preprocessing

- Convert to grayscale
- Apply histogram equalization

Improves contrast → better detection

3. Use ROI Constraints

- Detect only in expected regions (e.g., upper body area)

Reduces search space

Eliminates irrelevant regions

4. Switch to Better Models

- Replace Haar with:
 - DNN (ResNet SSD)
 - MTCNN

More robust feature learning

Handles variations better

5. Add Face Validation Step

- Verify detected region using:
 - Eye detection
 - Landmark detection

Confirms actual face structure

6. Apply Non-Maximum Suppression (NMS)

Removes overlapping duplicate detections

Conclusion

Combining **parameter tuning** + **better models** + **validation** + **preprocessing** significantly reduces false positives.

13. Minimal Face Recognition Pipeline

Objective

Build a simple system to:

- Detect face
- Recognize identity

Pipeline Design

CAPTURE image from camera

CONVERT to grayscale

DETECT faces

FOR each face:

EXTRACT ROI

RESIZE to fixed size

NORMALIZE image

RECOGNIZE using trained model

DISPLAY name + bounding box

Key Components

1. Face Detection

- Haar Cascade (fast)
- Used for ROI extraction

2. Preprocessing

- Resize (e.g., 100×100)
- Normalize pixel values

Ensures consistency

3. Recognition Model

- LBPH (Local Binary Pattern Histogram)

Lightweight

Works well with small datasets

4. Label Mapping

- ID → Name dictionary

Design Decisions

Component	Choice	Reason
Detector	Haar	Fast
Recognizer	LBPH	Simple & effective
Preprocessing	Resize + grayscale	Uniform input

Conclusion

A minimal pipeline uses **Haar + LBPH**, balancing **simplicity and performance** for demos.

14. Expression Recognition Fails with Head Movement

Problem Analysis

Failure occurs due to:

- Pose variation (rotation, tilt)
- Misaligned face
- Feature inconsistency
- Model trained on frontal faces only

Objective

Make system robust to slight head movements.

Proposed Improvements

1. Face Alignment (Most Important)

DETECT facial landmarks
ALIGN eyes horizontally
ROTATE face to standard orientation

Normalizes pose
Improves feature consistency

2. Use Robust Models

- CNN-based emotion classifier
- Trained on multi-angle datasets

Handles variations better

3. Data Augmentation

- Rotate images
- Flip horizontally

Improves generalization

4. Use Landmark-Based Features

- Extract distances between eyes, mouth, eyebrows

Less sensitive to rotation

5. Temporal Smoothing (for video)

```
emotion = average(last_n_predictions)
```

Reduces fluctuations
Stabilizes output

6. Multi-View Detection

- Detect faces at different angles

Conclusion

Combining **face alignment + robust models + augmented training data + temporal smoothing** significantly improves performance under head movement.